

```
// ===== CLIENT =====
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;

class ChatClient extends JFrame implements ActionListener, Runnable,
MouseListener {
    private JTextArea chatArea;
    private JList<String> userList;
    private DefaultListModel<String> listModel;
    private Socket socket;
    private DataInputStream dis;
    private DataOutputStream dos;
    private String myNickname;
    private String selectedUser = null;
    private JTextField msgField;

    public ChatClient(String serverAddress, int port, String nickname) {
        this.myNickname = nickname;
        setTitle("☞ Chat - " + nickname);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(600, 400);
        setLocationRelativeTo(null);

        // GUI bileşenleri
        chatArea = new JTextArea();
        chatArea.setEditable(false);
        chatArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane chatScroll = new JScrollPane(chatArea);
        chatScroll.setPreferredSize(new Dimension(400, 300));

        listModel = new DefaultListModel<>();
        userList = new JList<>(listModel);
        userList.setFont(new Font("Dialog", Font.BOLD, 14));
        userList.addMouseListener(this); // ChatClient MouseListener implemente
etti

        JScrollPane userScroll = new JScrollPane(userList);
        userScroll.setPreferredSize(new Dimension(180, 300));

        // Mesaj gönderme alanı
        msgField = new JTextField();
        msgField.addActionListener(this);

        JButton sendBtn = new JButton("Gönder");
        sendBtn.addActionListener(this);

        JPanel bottomPanel = new JPanel(new BorderLayout());
        bottomPanel.add(msgField, BorderLayout.CENTER);
        bottomPanel.add(sendBtn, BorderLayout.EAST);

        // Layout
        JPanel mainPanel = new JPanel(new BorderLayout());
        mainPanel.add(chatScroll, BorderLayout.CENTER);
        mainPanel.add(userScroll, BorderLayout.EAST);
        mainPanel.add(bottomPanel, BorderLayout.SOUTH);

        add(mainPanel);

        // Server'a bağlan
        try {
            socket = new Socket(serverAddress, port);
            dis = new DataInputStream(socket.getInputStream());

```

```

        dos = new DataOutputStream(socket.getOutputStream());

        // Nickname gönder
        dos.writeInt(1);
        dos.writeUTF(nickname);
        dos.flush();

        // Server'dan gelen mesajları dinle
        Thread listenerThread = new Thread(this);
        listenerThread.start();

    } catch (IOException e) {
        JOptionPane.showMessageDialog(this, "Server bağlantısı başarısız!");
        System.exit(1);
    }

    setVisible(true);
}

// ===== MouseListener metotları =====

// Fare tıklandığında çağrılır
public void mouseClicked(MouseEvent e) {
    if (e.getClickCount() == 2) { // Çift tıklama mı?
        selectedUser = userList.getSelectedValue();
        if (selectedUser != null && !selectedUser.equals(myNickname)) {
            chatArea.append("\n--- " + selectedUser + " ile sohbet
başlatıldı ---\n");
        }
    }
}

// Fare basıldığında çağrılır (kullanmıyoruz ama implement etmek zorundayız)
public void mousePressed(MouseEvent e) {
    // Bu metodu kullanmıyoruz
}

// Fare bırakıldığında çağrılır (kullanmıyoruz ama implement etmek
zorundayız)
public void mouseReleased(MouseEvent e) {
    // Bu metodu kullanmıyoruz
}

// Fare alana girdiğinde çağrılır (kullanmıyoruz ama implement etmek
zorundayız)
public void mouseEntered(MouseEvent e) {
    // Bu metodu kullanmıyoruz
}

// Fare alandan çıktığında çağrılır (kullanmıyoruz ama implement etmek
zorundayız)
public void mouseExited(MouseEvent e) {
    // Bu metodu kullanmıyoruz
}

// ===== ActionListener metodu =====
public void actionPerformed(ActionEvent e) {
    sendMessage(msgField.getText());
}

// ===== Runnable metodu =====
public void run() {
    try {
        while (true) {

```

```

        String msg = dis.readUTF();
        processServerMessage(msg);
    }
} catch (IOException e) {
    System.out.println("❗ Bağlantı koptu.");
    chatArea.append("\n*** Server ile bağlantı kesildi ***\n");
}
}

private void processServerMessage(String msg) {
    if (msg.startsWith("LIST:")) {
        String nicksStr = msg.substring(5);
        listModel.clear();
        if (!nicksStr.isEmpty()) {
            String[] nicks = nicksStr.split(",");
            for (int i = 0; i < nicks.length; i++) {
                listModel.addElement(nicks[i]);
            }
        }
    } else if (msg.startsWith("WELCOME:")) {
        chatArea.append(msg.substring(8) + "\n");
    } else if (msg.startsWith("MSG:")) {
        String content = msg.substring(4);
        chatArea.append(content + "\n");
    } else if (msg.startsWith("ERROR:")) {
        JOptionPane.showMessageDialog(this, msg.substring(6));
        System.exit(1);
    }
}

private void sendMessage(String text) {
    if (text.trim().isEmpty() || selectedUser == null) {
        if (selectedUser == null) {
            JOptionPane.showMessageDialog(this, "Lütfen önce kullanıcı listesinden birine çift tıklayın!");
        }
        return;
    }

    try {
        String fullMsg = selectedUser + ":" + text;
        dos.writeInt(2);
        dos.writeUTF(fullMsg);
        dos.flush();
        chatArea.append("(Siz) " + text + "\n");
        msgField.setText("");
    } catch (IOException e) {
        e.printStackTrace();
    }
}

public static void main(String[] args) {
    String nickname = JOptionPane.showInputDialog("Nickname girin:");
    if (nickname == null || nickname.trim().isEmpty()) {
        System.out.println("Nickname gerekli!");
        System.exit(1);
    }
    new ChatClient("localhost", 12348, nickname.trim());
}
}

```